

AI Website Security Grader [V1]

Eliška Jelinek

March 2026

Introduction

Background and Motivation

AI is the hot topic these days. Nearly everyone is hopping on the trend, and relishing at the plethora of possibilities that are now within their reach. Web development has been particularly affected. As it turns out, it doesn't take a genius to build a website these days. Just a reasonably intelligent AI tool. While this is wonderful in terms of making the internet and web development more accessible to the average person, it comes with its issues.

Meet Lovable, a Swedish vibe coding app that “allows customers without any technical training to instantly create websites and apps using only natural language prompts [1]. An employee at Replit recently found a security vulnerability present in many sites created with Lovable that allowed “anyone to access information about the [customer created] site's users” and their personal information, including their financial information and secret API keys [1]. This vulnerability is just one of many that highlights a new issue in the age of AI with growing capabilities: “each new app or website created by novices is a potential sitting duck for hackers” [1].

Research is being conducted by several teams to identify the most common security failings of AI generated code. So far, they've found LLM tools take optimization shortcuts that don't pay attention to the larger security context (like using overly powerful functions), produce code that blatantly omits security practices (encoding, validation, etc), and write subtle logic errors [2]. Other risks stem from AIs being overly permissive by default, producing more (unnecessary) code and thus more attack surface, a lack of security context, hallucinated dependencies, authentication and authorization failures, injection flaws and missing input validation [3, 8]. Many of these issues come from the “repetition of insecure patterns from training data”. SQL injection for example, is an issue, because textbook examples of SQL queries include flaws (are susceptible to SQL injection). [2]

The failings are already presenting themselves in the code that is being produced and published on the internet. Using CodeQL static analysis, Schreiber and Tippe developed a

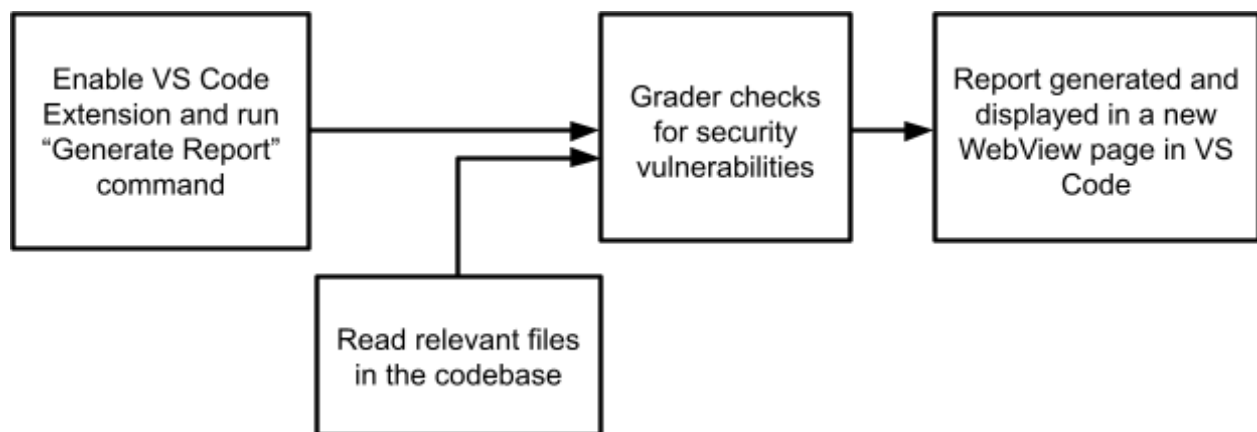
comprehensive empirical analysis of security vulnerabilities in AI-generated code across public GitHub repositories. They found “substantial differences in vulnerability profiles across programming languages with Python consistently exhibiting higher vulnerability rates than JavaScript and TypeScript” [5]. Another study analyzed code snippets produced by Github Copilot and other AI code generation models and “identified 733 snippets, revealing a high likelihood of security weaknesses, with 29.5% of Python and 24.2% of JavaScript snippets affected. These issues span 43 Common Weakness Enumeration (CWE) categories, including significant ones like CWE-330: Use of Insufficiently Random Values, CWE-94: Improper Control of Generation of Code, and CWE-79: Cross-site Scripting” [6]. Yet another study found that LLMs struggle with generating secure code, especially in regard to backend development. “Our experiments reveal key limitations of current LLMs in both functionality and security: (i) even the best model, OpenAI o1, achieves a mere 62% on code correctness; (ii) on average, we could successfully execute security exploits on around half of the correct programs generated by each LLM; and (iii) in less popular backend frameworks, models further struggle to generate correct and secure applications.” [10]. The evidence doesn’t stop there. A large scale study evaluated over 500k code samples, in both Python and Java. They classified security vulnerabilities using CWE and found that “on average, compared to human code, AI models generate more vulnerable code both in Python (+5k samples) and Java (+18k samples)” [9].

The problem isn’t just that AI is producing insecure code. It’s also that people *believe it’s good code*. A study looked into this phenomenon and discovered that not only did participants with access to an AI assistant write more insecure code, they also “were more likely to believe they wrote secure code than those without access to the AI assistant”. [4] This false confidence is dangerous, and needs to be addressed before it leads to serious damage. While it’s clearly established that there is a problem, it’s not clear as to who the responsibility of fixing it lies upon. Are the companies who provide the AI tools the ones who should also take the blame for security vulnerabilities? Or is it the developer, however amateur they may be, who is responsible for implementing secure practices?

Overview

I aim to tackle the issue of AI producing insecure code in web development by building a VS Code extension that grades a basic website codebase on its implementation of standard security practices. The grader will check for known security malpractices and give the user an overall score on the safety of their website. The report will also include details about what specifically the user needs to update in order to avoid security vulnerabilities.

The overall flow of the system is as follows:



The Project Itself

Security Practices

The aim of this project is to grade a basic website's adherence to standard security practices. For now, the extension is going to test for issues that can be checked using hard coded regex pattern matching. The rules are defined as follows:

```
function loadSecurityRules(): SecurityRule[] {
  return [
    // --- Hardcoded Secrets ---
    new SecurityRule(/password\s*=\s*["'].+["']/,      ['.js', '.ts', '.py', '.java'], 10,
    "Hardcoded password"),
    new SecurityRule(/apiKey\s*=\s*["'].+["']/,        ['.js', '.ts', '.py', '.java'], 8,
    "Hardcoded API key"),
    new SecurityRule(/secret\s*=\s*["'].+["']/,        ['.js', '.ts', '.py', '.java'], 8,
    "Hardcoded secret"),
    new SecurityRule(/token\s*=\s*["'].+["']/,         ['.js', '.ts', '.py', '.java'], 8,
    "Hardcoded token"),
    new SecurityRule(/private_key\s*=\s*["'].+["']/,   ['.js', '.ts', '.py', '.java'], 10,
    "Hardcoded private key"),

    // --- XSS (Cross-Site Scripting) ---
    new SecurityRule(/\.innerHTML\s*=/,                ['.js', '.ts', '.html'], 8, "Direct
    innerHTML assignment -- dangerous if value contains user input"),
    new SecurityRule(/document\.write\s*\(/,           ['.js', '.ts', '.html'], 7,
    "document.write() is a common XSS vector"),
    new SecurityRule(/eval\s*\(/,                      ['.js', '.ts'], 9, "eval()
    executes arbitrary code -- high injection risk"),
    new SecurityRule(/dangerouslySetInnerHTML/,        ['.js', '.ts', '.jsx', '.tsx'], 7,
    "dangerouslySetInnerHTML bypasses React's XSS protection"),

    // --- SQL Injection ---
    new SecurityRule(/"SELECT.*\+/,                    ['.js', '.ts', '.py', '.java'], 9, "SQL query built
    with string concatenation -- risk of SQL injection"),
    new SecurityRule(/`SELECT.*\${/,                   ['.js', '.ts'], 9, "SQL query with
    template literal interpolation -- risk of SQL injection"),

    // --- Insecure Connections ---
    new SecurityRule(/http:\/\/(?!\localhost)/,        ['.js', '.ts', '.py', '.html', '.java'],
    5, "HTTP used instead of HTTPS -- data is sent unencrypted"),

    // --- Command Injection ---
    new SecurityRule(/require\s*\(\s*["']child_process["']\s*\)/, ['.js', '.ts'], 7,
    "Importing child_process -- ensure user input never reaches exec/spawn"),
    new SecurityRule(/\.exec\s*\(/,                    ['.js', '.ts', '.py'], 7, "exec() can run shell
    commands -- dangerous with user-controlled input"),
    new SecurityRule(/Runtime\.getRuntime\(\)\.exec\(/,  ['.java'], 7, "Runtime.exec() can
    lead to command injection"),
```

```

// --- Weak Cryptography ---
new SecurityRule(/Math\.random\s*\(/, ['.js', '.ts'], 3, "Math.random() is not
cryptographically secure -- use crypto.getRandomValues() instead"),
new SecurityRule(/md5\s*\(/, ['.js', '.ts', '.py'], 6, "MD5 is
cryptographically broken -- use SHA-256 or stronger"),
new SecurityRule(/sha1\s*\(/, ['.js', '.ts', '.py'], 6, "SHA-1 is
cryptographically broken -- use SHA-256 or stronger"),

// --- Developer Oversights ---
new SecurityRule(/TODO.*security/i, ['.js', '.ts', '.py', '.java', '.html'], 2,
"Unresolved security TODO comment"),
new SecurityRule(/\\/\\s*password/i, ['.js', '.ts'], 5,
"Commented-out password in source code"),
];
}

```

As seen above, the rules fall into one of the following categories: hardcoded secrets, XSS (cross-site scripting), SQL injection, insecure connections, command injection, weak cryptography, and developer oversights. Each of these malpractices leave room for bad actors to hack into the website and gain access to potentially sensitive and personal information. As such, it is essential that no published website on the internet is vulnerable to these most basic of attacks, especially since these are all very well known vulnerabilities.

Not all vulnerabilities are created equal, however, which is why they each have their own individual “severity” ranking. I assigned these numbers based on how critical of a situation the user would be in if a bad actor were to take advantage of the vulnerability. For example, hardcoded password information would give a bad actor keys to the kingdom, whereas a comment about a forgotten “todo” item is unlikely to be harmful (especially since the other rules are already checking for issues).

Grading Algorithm

As mentioned above, every issue is given a severity ranking (1-10; 1 low, 10 high) based on the danger of the problem. The grading algorithm then sums all the severity rankings of the found issues and gives the user a grade based on how many “severity points” they received. The fewer issues found, the lower the severity score, the higher the grade. The grading algorithm returns an overall letter grade, where A is high and F is low, just like standard grading practices.

```
// main entry point: scans all files and returns the grade + full issue list
export function calculateGrade(filePaths: string[]): { grade: string, issues:
SecurityIssue[] } {
    const rules = loadSecurityRules();
    const issues = findIssues(filePaths, rules);

    // sum severity scores across all issues to determine overall grade
    const totalSeverity = issues.reduce((sum, issue) => sum + issue.severity, 0);

    let grade: string;
    if (totalSeverity === 0) {
        grade = 'A';
    } else if (totalSeverity <= 20) {
        grade = 'B';
    } else if (totalSeverity <= 40) {
        grade = 'C';
    } else if (totalSeverity <= 60) {
        grade = 'D';
    } else {
        grade = 'F';
    }

    return { grade, issues };
}
```

VS Code Extension

The entry point for using the extension is the “Generate Report” command, available in the command palette in the Extension Development Host. A more clear walkthrough of how to set up and run a custom, unpublished, extension is available in the demo, linked in the “The System in Action” section below. Running the “Generate Report” command in the desired codebase (the workspace of the website the user would like to grade) opens a new webview panel with the report details.

```
export async function activate(context: vscode.ExtensionContext) {
    const disposable =
vscode.commands.registerCommand('websitesecuritygrader.generateReport', async () => {
    // find all workspace files, skipping build artifacts and dependencies
    const fileUriis = await vscode.workspace.findFiles(
        '**/*',
        '{**/node_modules/**,**/.git/**,**/out/**,**/dist/**}'
    );
});
```

```

const filePaths = fileUriis.map(uri => uri.fsPath);

// run the regex-based scan
const result = calculateGrade(filePaths);

// open the webview and render the report
const panel = vscode.window.createWebviewPanel(
    'report',
    'Codebase Security Report',
    vscode.ViewColumn.One,
    {}
);
panel.webview.html = getReportHtml(result);

vscode.window.showInformationMessage(`Scan complete -- Grade: ${result.grade}`);
});

context.subscriptions.push(disposable);
}

```

The overall flow of the extension is as follows: the extension registers the “Generate Report” command so that it knows to run when the user executes it in the VS Code command palette, reads all of the relevant files in the open workspace, checks for issues in the code, calculates the code’s grade using the grading algorithm, and finally opens up a webview panel with HTML clearly displaying the report’s results.

Demonstration

The System in Action

Click the link below to view a demonstration of the grader being used on two sample websites, one with minimal issues and another with minimal adherence to safe practices.

[Video Demo](#)

The Code

View the full project here: [GitHub Repository](#)

A note about AI use: AI was used in the development of this project, primarily for writing out the HTML components, the security rules, portions of the ReadMe, and the initial structure of the extension. I have reviewed and edited all AI-generated code to ensure it meets the project requirements and functions correctly. The AI was a helpful tool for quickly generating boilerplate code and reducing the too-large scope of this project, and all final code has been vetted and tested by me. I took the time to research VS Code extension development myself, as well as common web security vulnerabilities that I could easily test for with regex. I specifically used Claude. (Side note, even though AI was used to write some of the code, this report is entirely my own work.)

Conclusion

Next Steps and Reflection

The current implementation of this website security grader system has so much room for further improvements. The biggest next step would be to use AI to check for more complex vulnerabilities, rather than hard coded versions. There are many open source or publicly available models that are fully capable of scanning code for known malpractices. I actually did start looking into implementing this directly into my system, although I concluded that it was beyond the scope of this particular class project. Going about adding AI consists of four additions to the flow of the system: 1) configuration of the API access (I'd use an API key with Anthropic to start), 2) building out the service layer of code that interacts with the API, 3) prompt engineering clear and specific instructions sent to the model, and finally 4) handling the responses and parsing the feedback to be used in the report.

The system could also be expanded to check for more extensive malpractices. The AI implementation would help with this, although more detailed checks can be hard coded as well. I could also modify the system to work on more difficult codebases. So far, I've only tested it on basic ones because each additional file adds complexity that the current system may not be able to handle. Future implementations of this system could also directly help the user fix their security issues. It could help them take that next step after identifying the issues and actually help fix their code. Finally, the most obvious next step would be to actually publish the extension so that people in the real world can make use of it.

Even a basic version of this system could prove to be immensely helpful for the amateur coders out there, especially the ones relying on AI to make their websites for them. It's easy to overlook security concerns when coding, particularly when one doesn't even know to take them into consideration in the first place. I hope to take this system to the next level and help fellow coders code safely.

References

- [1] R. Albergotti, “The hottest new vibe coding startup may be a sitting duck for hackers,” *Semafor.com*, May 29, 2025.
<https://www.semafor.com/article/05/29/2025/the-hottest-new-vibe-coding-startup-lovable-is-a-sitting-duck-for-hackers> (accessed Feb. 23, 2026).
- [2] “Understanding Security Risks in AI-Generated Code | CSA,” *Cloudsecurityalliance.org*, 2025.
<https://cloudsecurityalliance.org/blog/2025/07/09/understanding-security-risks-in-ai-generated-code> (accessed Feb. 23, 2026).
- [3] “AI’s Hidden Security Debt,” *The Hacker News*, Aug. 18, 2025.
<https://thehackernews.com/expert-insights/2025/08/ais-hidden-security-debt.html> (accessed Feb. 23, 2026).
- [4] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do Users Write More Insecure Code with AI Assistants?,” *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2785–2799, Nov. 2023, doi: <https://doi.org/10.1145/3576915.3623157>.
- [5] M. Schreiber and P. Tippe, “Security Vulnerabilities in AI-Generated Code: A Large-Scale Analysis of Public GitHub Repositories,” *Lecture Notes in Computer Science*, pp. 153–172, Oct. 2025, doi: https://doi.org/10.1007/978-981-95-3537-8_9.
- [6] Y. Fu *et al.*, “Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study,” *arXiv.org*, 2023. <https://arxiv.org/abs/2310.02059> (accessed Feb. 23, 2026).
- [7] C. Xia and Alalfi, Manar H, “Taught by the Flawed: How Dataset Insecurity Breeds Vulnerable AI Code,” *arXiv.org*, 2025. <https://arxiv.org/abs/2511.09879> (accessed Feb. 23, 2026).
- [8] E. Labs, “The Most Common Security Vulnerabilities in AI-Generated Code,” *Endorlabs.com*, Aug. 12, 2025.

<https://www.endorlabs.com/learn/the-most-common-security-vulnerabilities-in-ai-generated-code> (accessed Feb. 23, 2026).

[9] D. Cotroneo, C. Improtà, and P. Liguori, “Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity,” *arXiv.org*, 2025.

<https://arxiv.org/abs/2508.21634> (accessed Feb. 23, 2026).

[10] M. Vero *et al.*, “BaxBench: Can LLMs Generate Correct and Secure Backends?,” *arXiv.org*, 2025. <https://arxiv.org/abs/2502.11844> (accessed Feb. 23, 2026).

[11] Ori Meidan, “Synthetic Vulnerabilities: Why AI-Generated Code is a Potential Structural Security Crisis,” *Radware.com*, Dec. 09, 2025.

<https://www.radware.com/blog/threat-intelligence/synthetic-vulnerabilities/> (accessed Feb. 23, 2026).

[12] “Cybersecurity Risks of AI-Generated Code | Center for Security and Emerging Technology,” *Center for Security and Emerging Technology*, Nov. 19, 2024.

<https://cset.georgetown.edu/publication/cybersecurity-risks-of-ai-generated-code/> (accessed Feb. 23, 2026).

[13] negarakorruptor, “When AI-Built Websites Look Impressive but Hide Serious Security Issues,” *Medium*, Jan. 15, 2026.

<https://medium.com/%40nurmayadiliana/when-ai-built-websites-look-impressive-but-hide-serious-security-issues-ed92679376b2> (accessed Feb. 23, 2026).